Task 1 :

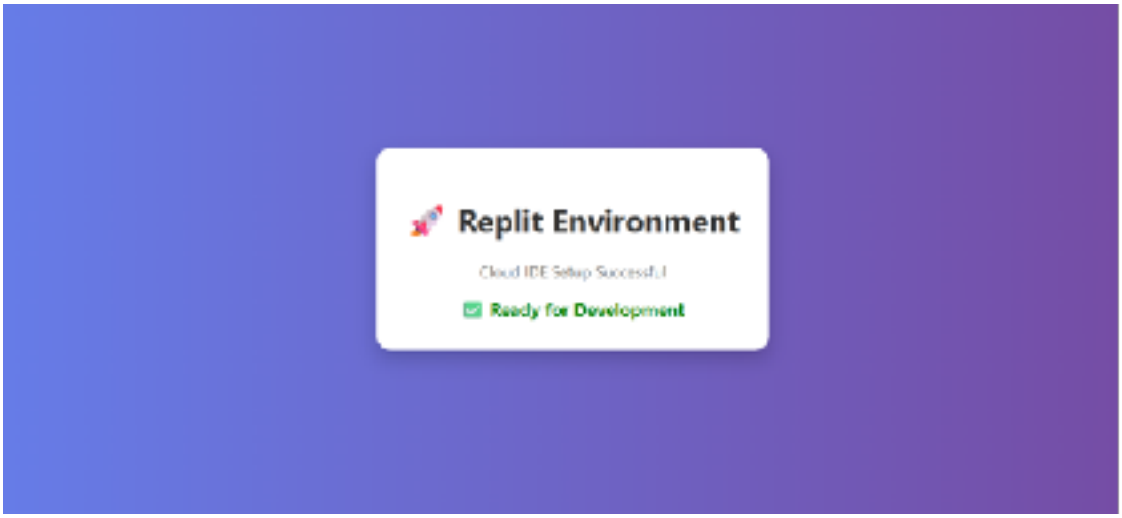
Prompt:

Set up a Replit environment for mobile application development using HTML5 or React-based templates. Install required dependencies and prepare the workspace for AI-assisted coding.

Code:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Replit Setup</title>

    <style>
        body {
            margin: 0;
            font-family: Arial, sans-serif;
            background: linear-gradient(to right, #667eea, #764ba2);
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }

        .card {
            background: white;
            padding: 30px;
            border-radius: 15px;
            text-align: center;
            box-shadow: 0 8px 20px rgba(0,0,0,0.2);
        }

        h1 {
            color: #333;
        }

        p {
            color: #666;
        }

        .status {
            margin-top: 15px;
            font-size: 18px;
            color: green;
            font-weight: bold;
        }
    </style>
</head>

<body>

<div class="card">
    <h1>🚀 Replit Environment</h1>
    <p>Cloud IDE Setup Successful</p>
    <div class="status">✅ Ready for Development</div>
</div>

</body>
</html>
```

Sample Input/Output:



Code Explanation:

1. This task sets up the development environment using Replit.
2. Replit is a cloud-based IDE that runs in the browser.
3. It supports mobile app development using web technologies.
4. Templates like HTML, React, or Flutter can be selected.
5. The basic test confirms the environment is working.
6. No local installation is required.
7. AI assistance in Replit helps generate code quickly.
8. This setup is required for further tasks.

Task 2 : Basic Mobile App

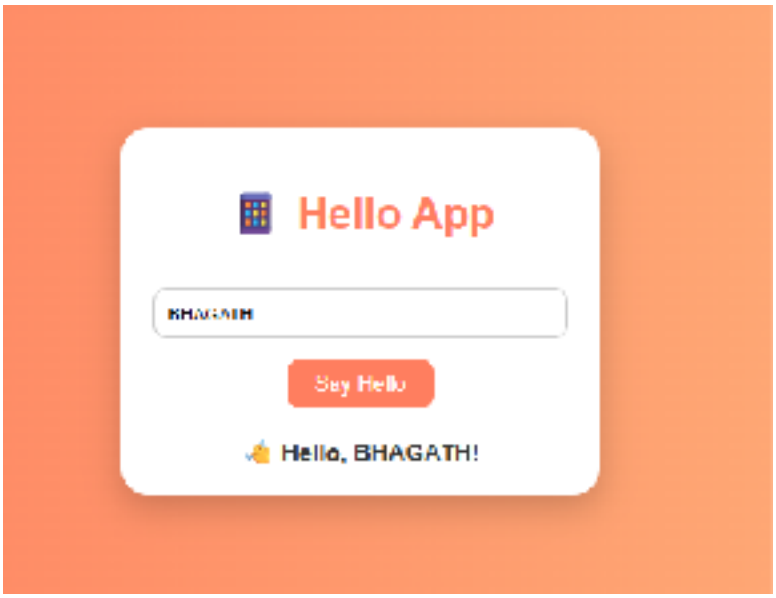
Prompt:

Build a simple mobile-style web app that displays an app title, takes user input, and shows a dynamic greeting message. Use AI assistance to generate colorful UI and interactive behavior.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Hello App</title>

    <style>
        body {
            margin: 0;
            font-family: Arial;
            background: linear gradient(to right, #ff7.5f, #fcb47b);
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }

        .app {
            background: white;
            width: 320px;
            padding: 30px;
            border radius: 20px;
            text-align: center;
            box-shadow: 0 10px 25px rgba(0,0,0,0.2);
        }

        h1 {
            color: #ff5722;
        }

        input {
            width: 100%;
            padding: 10px;
            margin-top: 15px;
            border-radius: 10px;
            border: 2px solid #ccc;
            outline: none;
        }

        button {
            margin-top: 15px;
            padding: 10px 20px;
            background: #ff5722;
            color: white;
            border: none;
            border radius: 10px;
            font size: 16px;
            cursor: pointer;
        }

        button:hover {
            background: #ff3333;
        }

        message {
            margin-top: 20px;
            font-size: 18px;
            color: #333;
            font-weight: bold;
        }
    </style>
</head>

<body>

<div class="app">
    <h1 ■ Hello App</h1>

    <input type="text" id="name" placeholder="Enter your name">

    <button onclick="showMessage()">Say Hello</button>

    <div id="message"></div>
</div>

<script>
function showMessage() {
    let name = document.getElementById("name").value;

    if(name === "") {
        document.getElementById("message").innerHTML = "⚠ Please enter your name!";
    } else {
        document.getElementById("message").innerHTML = "✨ Hello, " + name + "!";
    }
}
</script>

</body>
</html>
```

Sample Input/Output:



Code Explanation:

1. This task builds a simple interactive mobile-style app.
2. The UI is designed using HTML and CSS with a gradient background.
3. A centered card layout improves visual appearance.
4. The input field takes the user's name.
5. A button triggers a JavaScript function.
6. The function dynamically updates the greeting message.
7. Input validation ensures proper user interaction.
8. This demonstrates basic UI + logic integration.

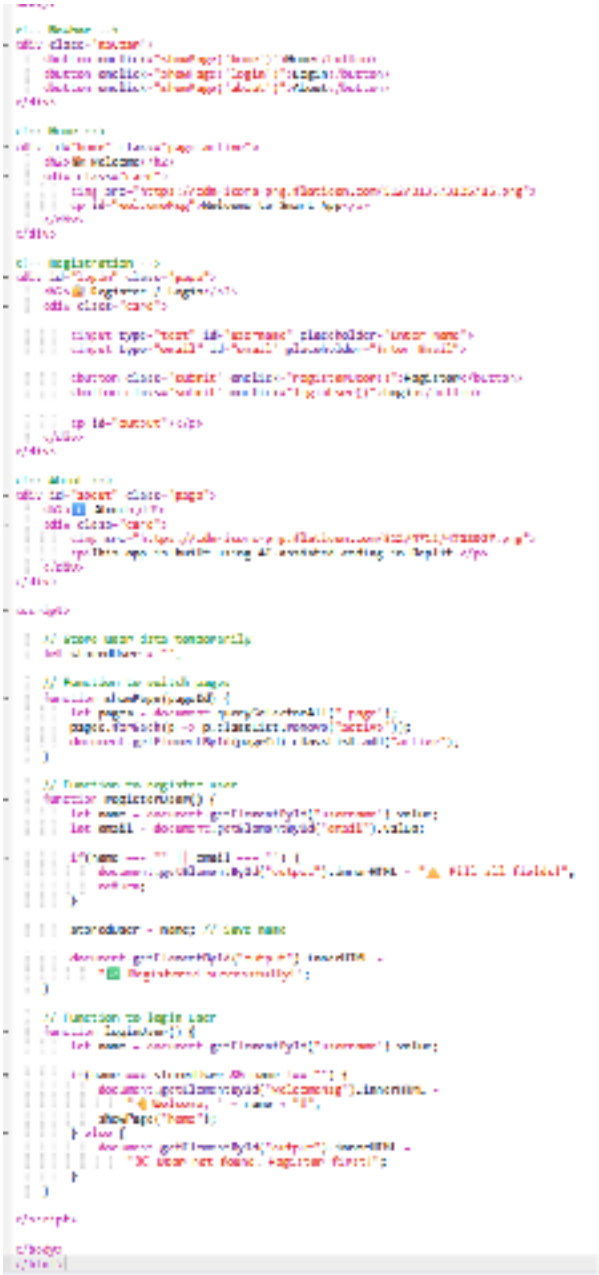
Task 3 : Adding Features with AI Assistance

Prompt:

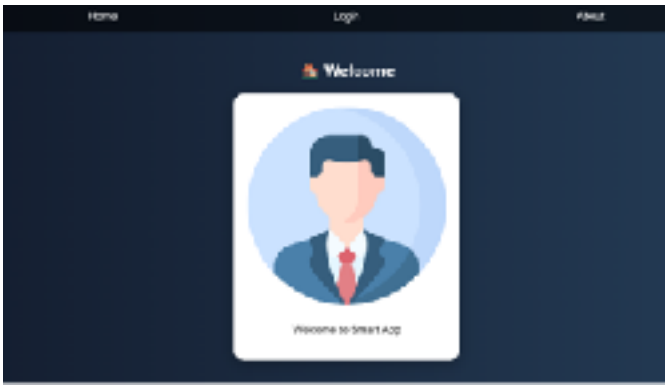
Enhance the mobile app by adding navigation, form handling, and improved styling using AI assistance. Include inline documentation for each function to improve readability.

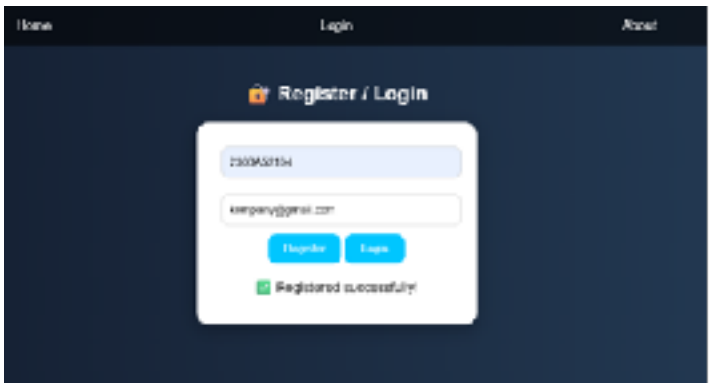
Code:

```
<!DOCTYPE html>
<html
<head
  <title>Mobile App</title>

  <style>
    body {
      margin: 0;
      font-family: Arial;
      background: linear-gradient(to right, [illegible], [illegible]);
      color: white;
    }

    /* Navbar */
    .navbar {
      display: flex;
      justify-content: space-around;
      background: rgba(0,0,0,0.5);
      padding: 15px;
    }

    .navbar button {
      background: none;
      border: none;
      color: white;
      font-size: [illegible];
      cursor: pointer;
    }

    .navbar button:hover {
      color: [illegible];
    }

    /* Page */
    .page {
      display: none;
      text-align: center;
      padding: 20px;
    }

    .active {
      display: block;
    }

    /* Card */
    .card {
      background: white;
      color: black;
      padding: 20px;
      margin: auto;
      width: [illegible];
      border-radius: 10px;
      box-shadow: 0 [illegible] rgba(0,0,0,0.1);
    }

    input {
      width: 90%;
      padding: 10px;
      margin: [illegible];
      border-radius: 15px;
      border: 1px solid [illegible];
    }

    button.save {
      padding: 20px 20px;
      background: [illegible];
      border: none;
      border-radius: 10px;
      color: white;
      cursor: pointer;
    }

    button.save:hover {
      background: [illegible];
    }

    img {
      width: 100%;
      border-radius: 15px;
      margin-bottom: 15px;
    }
  </style>
</head>

<body>
```



Sample Input/Output:





Code Explanation:

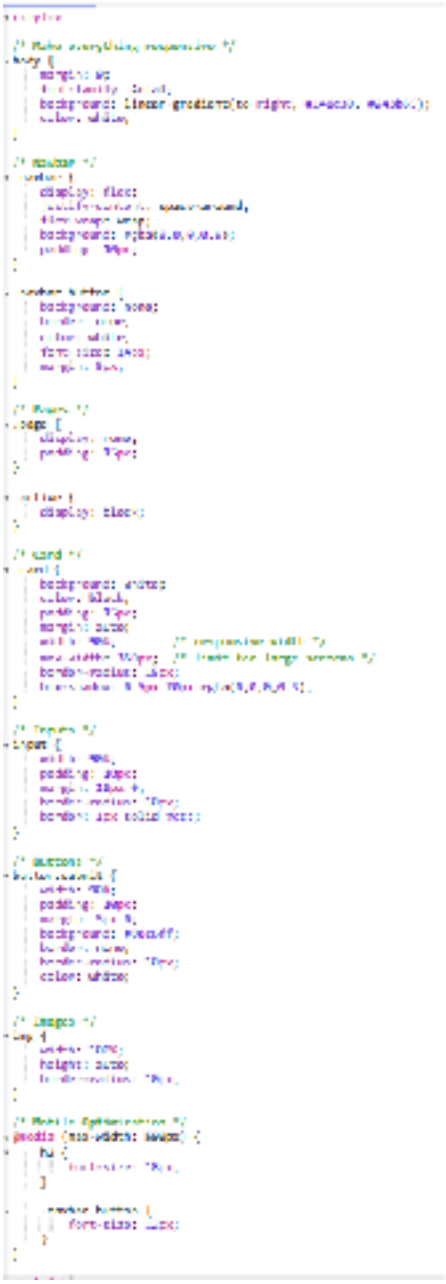
1. This task enhances the mobile app with additional features.
2. A navigation bar allows switching between pages.
3. JavaScript is used to control page visibility dynamically.
4. A registration form collects user details.
5. Input validation ensures correct data entry.
6. CSS improves UI with gradients and responsive layout.
7. Inline comments explain each function clearly.
8. This demonstrates multi-page app behavior in a single file.

Task 4 :

Prompt:

Test the mobile app in Replit's mobile preview mode and ensure it is responsive across different screen sizes. Fix UI alignment issues using AI suggestions and responsive design techniques.

Code:



Sample Input/Output:

Code Explanation:

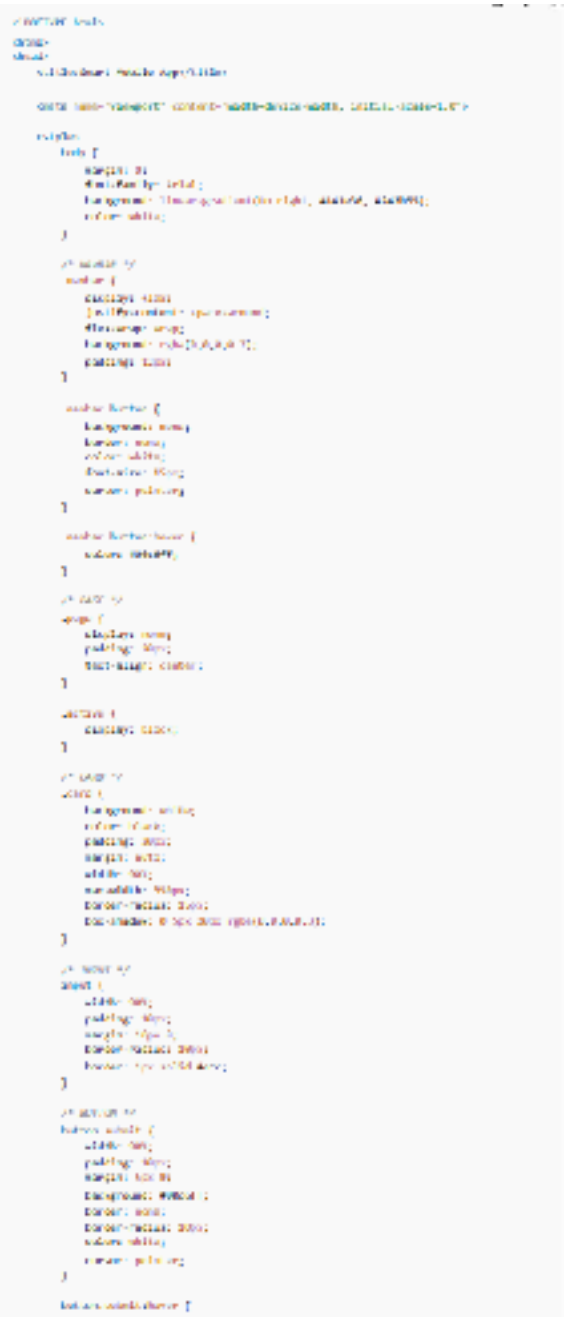
1. This task focuses on testing the application on mobile devices.
2. Replit provides a mobile preview for testing responsiveness.
3. The UI is checked across different screen sizes.
4. Issues like overflow and misalignment are identified.
5. CSS media queries are used to fix responsiveness.
6. Flexible layouts ensure proper display on all devices.
7. Images and buttons are resized dynamically.
8. This improves user experience across platforms.

Task 5 : Deployment

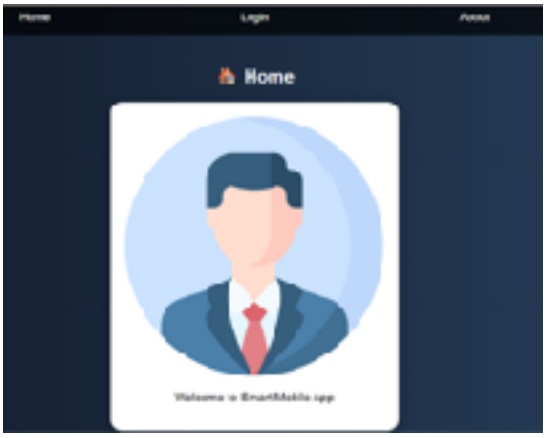
Prompt:

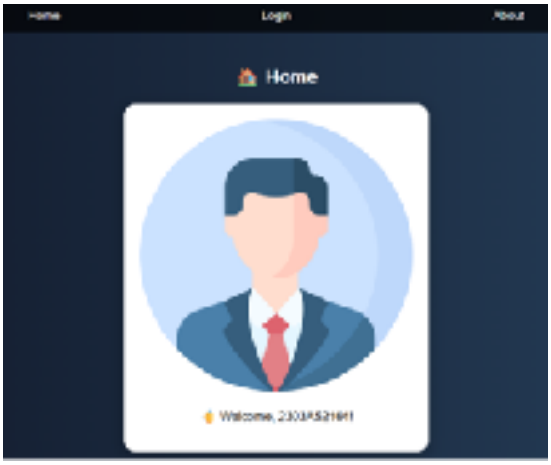
Deploy the developed mobile web application using Replit hosting and make it accessible via a public link. Test the deployed app and share the link for peer evaluation.

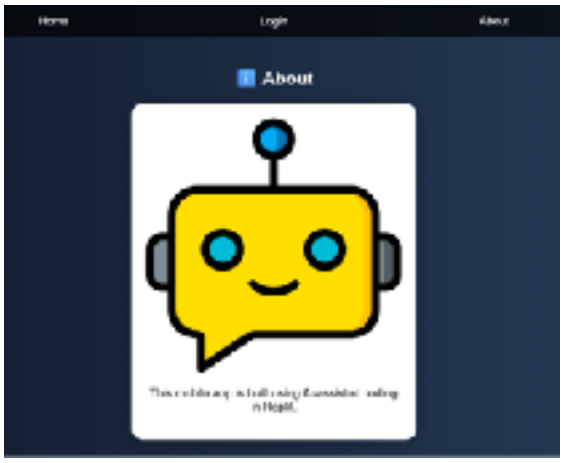
Code:



Sample Input/Output:







Code Explanation:

1. This task focuses on deploying the developed application.
2. Replit provides free hosting for web applications.
3. The application is made accessible via a public URL.
4. Users can test the app in real-time using the link.
5. Deployment ensures the app works outside the development environment.
6. It allows sharing with peers for feedback.
7. AI assistance helped speed up development and debugging.
8. This completes the full mobile app development lifecycle.